txt2tex Reference

Write math like you're at a whiteboard. Get LaTeX you can hand in.

https://github.com/jmf-pobox/txt2tex

Document Structure

Single-line directives.

`=== Title ===`	`\section *{Title}`
`** Solution N **`	Bold heading with spacing
`(a) Text`	Part label
`TITLE: My Doc`	Document title (`\title {}`)
`SUBTITLE: Sub`	Document subtitle
`AUTHOR: Name`	Author name
`DATE: 2026`	Date string
`INSTITUTION: Univ`	Institution line
`BIBLIOGRAPHY: f.bib`	BibTeX file
`BIBLIOGRAPHY_STYLE: plainnat`	Citation style
`CONTENTS:`	Table of contents
`PARTS: inline`	Parts formatting style
`PAGEBREAK:`	`\newpage`
`LINEBREAK:`	`\medskip` (gap, no new page)
`TEXT: prose here`	Smart prose (auto-detects formulas)
`PURETEXT: raw & text`	Verbatim prose; escapes LaTeX specials
`LATEX: \cmd`	Raw LaTeX passthrough (single-line)
`[cite key]`	`\citep {key}` (in TEXT blocks)

Identifier Decoration.

`count'`	$count'$ (after-state)
`in?`	$in?$ (input)
`out!`	$out!$ (output)
`x?'`	$x?'$ (combinations allowed)
`'sunk'`	'sunk' (string literal)

Decoration attaches to any identifier. Reserved words (`theta`, `defs`, `hide`, `project`, `Delta`, `Xi`) cannot be decorated.

Numbers & Arithmetic

`N`	$\mathbb{N}$ (natural numbers)
`Z`	$\mathbb{Z}$ (integers)
`N1`	$\mathbb{N}_1$ (positive naturals)
`n + m / n - m`	addition / subtraction
`n * m / n div m`	multiplication / integer division
`n mod m`	modulo
`n < m / n > m`	strict ordering
`n <= m / n >= m`	non-strict ordering
`n = m / n /= m`	equality / inequality

Exponentiation. The ^ operator is fuzz *iter*: it composes a relation with itself. It does NOT square a number. For powers of numbers, multiply directly.

You write:

```
R^3               (relation iterated three times)
x * x             (square of a number)
```

You get:

R^3 $x * x$

Propositional Logic

`p land q`	$p \land q$
`p lor q`	$p \lor q$
`lnot p`	$\neg\, p$
`p => q`	$p \Rightarrow q$
`p <=> q`	$p \Leftrightarrow q$

Note: Use `land`, `lor`, `lnot`. English `and`/`or`/`not` are not supported.

Truth-table example:

You write:

```
TRUTH TABLE:
p | q | p => q
T | T | T
T | F | F
F | T | T
F | F | T
```

You get:

p	q	$p \Rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Equivalence chain (`EQUIV:`):

You write:

```
EQUIV:
lnot (p land q)
lnot p lor lnot q  [De Morgan]
```

You get:

$$\begin{aligned} &\neg\,(p \land q) \\ &\Leftrightarrow \neg\, p \lor \neg\, q \qquad \text{[De Morgan]} \end{aligned}$$

`EQUIV:` and its alias `ARGUE:` join steps with $\Leftrightarrow$. Use for predicate equivalences.

Equality chain (`EQUAL:`):

You write:

```
EQUAL:
0 + 0
0      [0+0 = 0]
length(<>)
       [length(<>) = 0]
```

You get:

$$\begin{aligned} &0 + 0 \\ &= 0 && [0{+}0 = 0] \\ &= \mathit{length}(\langle\rangle) && [\mathrm{length}(\langle\rangle) = 0] \end{aligned}$$

`EQUAL:` joins steps with $=$. Use when steps are arithmetic/set-valued, not propositional equivalences.

Sets & Types

`x elem S`	$x \in S$
`x notin S`	$x \notin S$
`A subset B`	$A \subseteq B$
`A psubset B`	$A \subset B$ (proper subset)
`A union B`	$A \cup B$
`A intersect B`	$A \cap B$
`A setminus B`	$A \setminus B$
`bigcup S`	$\bigcup S$
`bigcap S`	$\bigcap S$
`power S`	$\mathbb{P}\, S$
`P1 S`	$\mathbb{P}_1\, S$ (non-empty)
`F S`	$\mathbb{F}\, S$
`F1 S`	$\mathbb{F}_1\, S$ (non-empty finite)
`A cross B`	$A \times B$
`{x : S | P}`	Set comprehension
`n..m`	$n \,..\, m$ (integer range)
`# S`	$\#S$ (cardinality / length)

Prefix-operator rule (#133): prefix-generic operators (`seq`, `P`, `dom`, `ran`, `bigcup`, ...) require an atomic argument. The engine automatically wraps a nested prefix call in parentheses: `seq (P X)` → `\seq~(\power X)`.

Set comprehension:

You write:

```
{x : N | x > 0 land x < 5}
```

You get:

$$\{x : \mathbb{N} \mid x > 0 \land x < 5\}$$

Predicate Logic

`forall x : S | P`	$\forall\, x : S \bullet P$
`exists x : S | P`	$\exists\, x : S \bullet P$
`exists_1 x : S | P`	$\exists_1\, x : S \bullet P$ (unique)
`lambda x : S | E`	$\lambda\, x : S \bullet E$
`mu x : S | P`	$\mu\, x : S \bullet P$ (definite desc.)
`if p then e1 else e2`	conditional expression

Bullet form: `x : S | P` means $x : S \bullet P$ (bullet is implied). The vertical bar is the quantifier separator.

Tuple-pattern:

You write:

```
forall (x, y) : N cross N |
  x + y >= 0
```

You get:

$$\forall (x, y) : \mathbb{N} \times \mathbb{N} \bullet x + y \geq 0$$

Multi-decl: separate variable groups with `;` inside a quantifier: `forall x : A; y : B | P`.

Schema-text scope: a bare `[d : T | P]` in a quantifier or function acts as an inline schema text.

Relations

`A <-> B`	$A \leftrightarrow B$ (relation type)
`x |-> y`	$(x \mapsto y)$ (maplet)
`dom R / ran R`	$\mathrm{dom}\, R$ / $\mathrm{ran}\, R$
`id`	identity relation
`inv R / R~`	R^{-1} (inverse; postfix or prefix)
`A dres R / A <| R`	$A \lhd R$ (domain restrict)
`A dsub R / A «| R`	$A \lhd\!\!\!- R$ (domain subtract)
`R rres B / R |> B`	$R \rhd B$ (range restrict)
`R rsub B / R |» B`	$R \rhd\!\!\!- B$ (range subtract)
`R oplus S / R ++ S`	$R \oplus S$ (relational override)
`R o9 S`	$R \fatsemi S$ (forward comp.)
`R comp S`	$R \fatsemi S$ (backward comp.)
`R(| A |)`	$R(\!\mid A \mid\!)$ (relational image)
`shows`	$\vdash$ (turnstile / judgment)

o9 vs comp: `o9` → `\semi` (forward, $(R; S)(x) = S(R(x))$). `comp` → `\comp` (backward, $(R \circ S)(x) = R(S(x))$).

Worked example — domain restrict:

You write:

```
{1, 2} dres {1 |-> a, 2 |-> b, 3 |-> c}
```

You get:

$$\{1, 2\} \lhd \{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\}$$

Functions

`A +-> B`	$A \pfun B$ (partial)
`A -> B`	$A \fun B$ (total)
`A >+> B / A -|> B`	$A \pinj B$ (partial injection)
`A >-> B`	$A \inj B$ (total injection)
`A +->> B`	$A \psurj B$ (partial surjection)
`A ->> B`	$A \surj B$ (total surjection)
`A >->> B`	$A \bij B$ (bijection)
`A 77-> B`	$A \ffun B$ (finite partial)
`f(x)`	function application
`lambda x :  T | f(x)`	$\lambda\, x : T \bullet f(x)$
`f ++ g`	$f \oplus g$ (override)
`f~`	f^{-1} (postfix inverse)

Worked example — lambda:

You write:

```
double == lambda n : N | n * 2
```

You get:

$$double == \lambda\, n : \mathbb{N} \bullet n * 2$$

Sequences & Bags

`<> / <a, b>`	$\langle\rangle$ / $\langle a, b\rangle$
`s ^ t`	$s \cat t$ (concat; space before `^` required)
`seq S / seq1 S`	$\mathrm{seq}\, S$ / $\mathrm{seq}_1\, S$
`iseq S`	$\mathrm{iseq}\, S$ (injective sequences)
`# s`	$\#s$ (length)
`head s / tail s`	first / rest
`last s / front s`	last / all-but-last
`rev s`	reverse
`s(i)`	index (1-based)
`s filter A`	$s \rhd A$ (filter)
`bag S`	$\mathrm{bag}\, S$
`[[a, b]]`	$[\![a, b]\!]$ (bag literal)
`b1 bag_union b2`	$b_1 \uplus b_2$
`b1 bag_diff b2`	$b_1 \setminus b_2$

Concat: write `<a> ^ <b>` (space before `^`). No space → relation iteration (`\bsup`). **bag_diff:** emits `\uminus` — bag subtraction clamped at zero.

Z Paragraphs

`given A, B`	Given types: $[A, B]$
`T ::= a | b`	Free type definition
`name == expr`	Abbreviation

`axdef` / `schema Name` / `gendef [X]`

`declarations`	Variable declarations
`where`	Separator
`predicates`	Constraining predicates
`end`	Close the block

`axdef` introduces global constants; **`gendef [X]`** parameterises by a type; **`schema Name`** names a state or operation.

Zed block (unboxed paragraph):

You write:

```
zed
  given Entry
  Status ::= active | inactive
  MaxSize == 100
end
```

You get:

$[Entry]$

$Status ::= active \mid inactive$

$MaxSize == 100$

`zed` blocks can mix given types, free types, abbreviations, and predicates in one environment. No visual box. Use for global type definitions; use **`axdef`**/**`schema`** for declarations with a **`where`** clause.

Syntax block (multi-type free definitions):

You write:

```
syntax
  OP ::= plus | minus | times
  Expr ::= const<N> | binop<OP cross Expr>
end
```

You get:

$$OP ::= plus \mid minus \mid times$$
$$Expr ::= const\,\mathbb{N} \mid binop\,OP \times Expr$$

`syntax` produces a column-aligned fuzz `syntax` environment. Blank lines between definitions emit `\also` (visual group separator).

Schemas

A named schema introduces a signature (declarations) and an invariant (where clause).

You write:

```
schema Counter
  size : N
  nums : seq N
where
  size = # nums
end
```

You get:

Counter

$size : \mathbb{N}$

$nums : \operatorname{seq} \mathbb{N}$

$size = \#nums$

Where-clause patterns:

1. Simple comparison.

You write:

```
schema Bounded
  size : N
where
  size < 10
end
```

You get:

Bounded

$size : \mathbb{N}$

$size < 10$

2. Set comprehension.

You write:

```
schema Inventory
  items : seq N
  stock : N
where
  stock = # {x : ran items | x > 0}
end
```

You get:

Inventory

$items : \operatorname{seq} \mathbb{N}$

$stock : \mathbb{N}$

$stock = \#\{\, x : \operatorname{ran} items \mid x > 0 \,\}$

3. Quantifier.

You write:

```
schema NonNeg
  s : seq Z
where
  forall i : 1..#s | s(i) >= 0
end
```

You get:

NonNeg

$$s : \operatorname{seq} \mathbb{Z}$$

$$\forall i : 1 \mathinner{..} \#s \bullet s(i) \geq 0$$

Generic schemas.

You write:

```
gendef [X]
  empty : F X
where
  empty = {}
end
```

You get:

$[X]$

$$empty : \mathbb{F}\ X$$

$$empty = \{\}$$

$[X]$ is the type parameter; instantiate at use with `empty[T]`.

Schemas as predicates.

Inside a quantifier, a schema name acts as a predicate (both its signature and its where clause must be satisfied):

You write:

```
forall c : Counter |
  c.size >= 0
```

You get:

$$\forall c : Counter \bullet c.size \geq 0$$

Schema Operations

Inclusion operators.

`SName`	bare inclusion
`Delta Counter`	$\Delta Counter$ (before/after-state pair)
`Xi Card`	$\Xi Card$ (read-only state)
`theta S`	θS (binding of current state)
`theta S'`	$\theta S'$ (binding of after-state)

Horizontal definition.

`Name defs RHS`:	
`Name defs Schema`	$Name \mathrel{\widehat=} Schema$
`N defs Delta C`	$N \mathrel{\widehat=} \Delta C$
`N defs [d : T | P]`	inline schema text
`N[X] defs G[X]`	generic form

Schema calculus operators (RHS of `defs`).

`S[old/new]`	rename component
`S[a/b, c/d]`	multiple renames
`S ; T`	$S \fatsemi T$ (composition)
`S » T`	$S \gg T$ (piping)
`S hide (x, y)`	$S \setminus (x, y)$
`S project T`	$S \upharpoonright T$

Z Bindings

`{| name == e |}`	$\langle name == e \rangle$
`{| a == e, b == f |}`	multiple components
`{| |}`	empty binding

Multi-typed comprehension with binding:

You write:

```
{ t : Track; a : Album |
  t.albumId = a.albumId .
  {| trackId == t.trackId |} }
```

You get:

$$\{t : Track; a : Album \mid t.albumId = a.albumId \bullet \langle trackId == t.trackId \rangle\}$$

Use `;` to separate variable-type pairs inside `{ }`. Binding components are **comma**-separated.

Fuzz note: binding expressions emit outside any Z environment — fuzz silently skips them. Do not wrap bindings inside `zed`/`axdef`/`schema` blocks.

Proofs

INFRULE: — named inference rule display:

You write:

```
INFRULE:
premise_1
premise_2
---
conclusion [Rule name]
```

You get:

$$\frac{premise_1 \quad premise_2}{conclusion}\ \text{Rule name}$$

`INFRULE:` renders a labelled rule schema. Use for displaying rule definitions (not proofs). The three-dash line `--` separates premises from conclusion.

How proof trees work (`PROOF:`):

The source is **conclusion-first** (rendered tree reads bottom-up). Premises are indented under their conclusion. A rule with two or more premises requires `::` on each premise — without it, indented lines collapse into a *linear chain* (each line becomes the sole premise of the line above) and the extra premises are silently dropped. Unary rules take a single indented child with no `::`.

Proof tree syntax:

indent	Premises indented under conclusion
`[rule]`	Justification label
`[rule from` n`]`	Discharge assumption n
`[`n`] expr [assumption]`	Boxed assumption labelled n
`expr [from` n`]`	Leaf reference to assumption n
`::`	Premise of a multi-premise rule
`case` X`:`	Branch in a **`lor elim`** case analysis

Rules: `land intro`, `land elim 1/2`, `lor intro 1/2`, `lor elim`, `=> intro/elim`, `<=> intro/elim`, `lnot intro/elim`, `forall intro/elim`, `exists intro/elim`.

Modus ponens (binary — requires `::` on each premise):

You write:

```
PROOF:
q [=> elim]
  :: p => q
  :: p
```

You get:

$$\frac{p \Rightarrow q \quad p}{q}\ \Rightarrow\text{-elim}$$

∧-intro (binary):

You write:

```
PROOF:
p land q [land intro]
  :: p
  :: q
```

You get:

$$\frac{p \quad q}{p \wedge q}\ \wedge\text{-intro}$$

⇒-intro with discharge (use `from N`):

You write:

```
PROOF:
p => p [=> intro from 1]
  [1] p [assumption]
  :: p [from 1]
```

You get:

$$\frac{\ulcorner p \urcorner^{[1]}}{p \Rightarrow p}\ \Rightarrow\text{-intro}^{[1]}$$

`[1] p [assumption]` declares the discharge binder. `p [from 1]` marks every leaf use. `[=> intro from 1]` discharges it.

∨-elim / case analysis (ternary — disjunction + two cases):

You write:

```
PROOF:
(p lor q) => r [=> intro from 1]
  [1] p lor q [assumption]
  :: r [lor elim from 1]
    :: p lor q [from 1]
    case p:
      :: r [...derive r from p...]
    case q:
      :: r [...derive r from q...]
```

You get:

$$\frac{p \vee q \quad \dfrac{\ulcorner p \urcorner^{[1]}}{r} \quad \dfrac{\ulcorner q \urcorner^{[1]}}{r}}{r}\ \vee\text{-elim}$$

`lor elim from N` discharges the disjunction's assumption [N]; inside each **`case X:`** block, the case formula is referenced as **`X [from N]`**.

Relational Database Notation

Primary key annotation.

Mark a primary-key attribute with `pk` in a named schema body. The annotation sits outside the Z box so fuzz type-checks cleanly.

You write:

```
schema Book
  pk bookId : BookId
  isbn      : ISBN
  pages     : N
end
```

You get:

Book

$$bookId : BookId$$
$$isbn : ISBN$$
$$pages : \mathbb{N}$$

$$\mathrm{PK}(\mathrm{Book}) = \{bookId\}$$

Composite: two `pk` lines $\Rightarrow \mathrm{PK}(S) = \{a, b\}$. Comma-separated names on one `pk` line also work. `pk` is rejected in `axdef`/`gendef`/inline schema text.

Relational algebra.

`sigma[p](R)`	$\mathrm{Restrict}_p(R)$ — restrict
`pi[A,B](R)`	$\mathrm{Project}_{A,B}(R)$ — project
`R[B/A]`	$R[B/A]$ — rename
`R join S`	$\mathrm{Join}(R, S)$ — natural join
`R join [p] S`	$\mathrm{Join}_p(R, S)$ — theta-join
`R div S`	$R \,\mathsf{div}\, S$ — division

Fuzz note: algebra expressions emit outside any Z environment — fuzz silently skips them. Do not wrap algebra inside `zed`/`axdef`/`schema` blocks.

FK predicate form.

Foreign-key constraints are expressed as axdef predicates:

You write:

```
axdef
where
  (R, S) |-> {a |-> b} elem FK
end
```

You get:

$$(R, S) \mapsto \{a \mapsto b\} \in FK$$

GROUP & UNGROUP.

Date's nested-relation operators.

`R group ({A,B} as x)`	R GROUP$(\{A, B\}$ AS $x)$
`R ungroup x`	R UNGROUP x

You write:

```
Members group ({name,email} as contact)
Members ungroup contact
```

You get:

$$Members \text{ GROUP } (\{name, email\} \text{ AS } contact)$$
$$Members \text{ UNGROUP } contact$$

Fuzz note: GROUP and UNGROUP expressions emit outside any Z environment — fuzz silently skips them. Do not wrap GROUP/UNGROUP inside `zed`/`axdef`/`schema` blocks.

GROUP aggregator form.

The aggregator keywords are `Count`, `Sum`, `Avg`, `Min`, `Max`, `Median`. Each takes one attribute name and an alias. The regroup form (`{...} as alias`) and the aggregator form are mutually exclusive in a single `group` expression.

`R group (Count(x) as n)`	$R\,\mathrm{Group}(\mathrm{Count}(x)\,\mathrm{as}\,n)$
`R group (Sum(x) as n)`	$R\,\mathrm{Group}(\mathrm{Sum}(x)\,\mathrm{as}\,n)$
`R group (Avg(x) as n)`	$R\,\mathrm{Group}(\mathrm{Avg}(x)\,\mathrm{as}\,n)$
`R group (Min(x) as n)`	$R\,\mathrm{Group}(\mathrm{Min}(x)\,\mathrm{as}\,n)$
`R group (Max(x) as n)`	$R\,\mathrm{Group}(\mathrm{Max}(x)\,\mathrm{as}\,n)$
`R group (Median(x) as n)`	$R\,\mathrm{Group}(\mathrm{Median}(x)\,\mathrm{as}\,n)$

You write:

```
Sales group (Count(orderId) as orderCount,
             Sum(amount) as totalRevenue)
```

You get:

$$Sales\,\mathrm{Group}(\mathrm{Count}(orderId)\,\mathrm{as}\,orderCount,\ \mathrm{Sum}(amount)\,\mathrm{as}\,totalRevenue)$$

Multiple aggregators are comma-separated. Each accepts exactly one attribute identifier — no nested aggregators, no compound expressions.

Quick reference.

`pk a :  T`	primary key in schema
`pk a, b :  T`	composite pk
`sigma[p](R)`	restrict
`pi[A](R)`	project
`R[B/A]`	rename attr (NEW/OLD)
`R join S`	natural join (Join(R, S))
`R div S`	division
`{| a == e |}`	binding bracket
`{S; C | P . E}`	multi-typed set comp
`R group ({A} as x)`	regroup attrs
`R group (Count(a) as n)`	aggregate
`R ungroup x`	ungroup attr